



廣州工商學院

Guangzhou College of Technology and Business

实 验 报 告

课 程 名 称: SpringBoot 企业级开发

实 验 项 目: 企业员工管理系统 (EMS)

专 业 班 级: 25 专升本软件工程 1 班

姓 名: 吴宇威

学 号: 202516060143

日 期: 2026 年 6 月 28 日

实践教学中心制

实验报告说明

1. 实验报告应包含实验目的、实验条件、实验方法、实验内容、结果分析、成绩评定等内容。
2. 学生应认真撰写实验内容，并对实验进行总结分析。
3. 指导教师应严格依照成绩评定标准对实验报告进行成绩评定并签字，评语围绕实验报告内容，具体、有针对性、有建设性。
4. 格式要求：A4 幅面，双面打印。**封面：**汉字采用三号宋体、外文数字采用三号 Times New Roman 体，下划线居中填写，下划线整体长度不改变；**正文：**汉字采用小四号宋体、外文数字采用小四号 Times New Roman 体；左对齐填写，首行缩进 2 字符，行距固定值 20 磅，图或表的字体大小为五号字。

实验地点：	实 II -B202	指导教师：	井煜
同组人员：	黄显杰		
一、实验目标 开发一套基于 SpringBoot 与 Vue 的前后端分离架构系统，题目可自行拟定。后端整合 MyBatis，以达成控制层、业务层和数据层的分离；前端运用 ElementPlus 等组件开展前端开发工作。主要目标包含以下方面： 1. 通过综合项目巩固并加深 SpringBoot 开发的相关知识，提升实际应用开发能力。 2. 培养学生的团队协作意识，提高项目开发过程中的沟通、协调及任务分工能力。 3. 锻炼学生在需求分析、功能设计、程序实现和测试调试等项目全流程的开发能力。 4. 培养学生的创新思维和解决实际问题的能力，丰富项目实践经验。 5. 让学生掌握规范化的软件开发流程，提高项目文档编写和代码质量管理能力。			
二、实验条件（环境） 1. Windows10、Win11、Mac 系统 2. IntelliJ IDEA 3. MySQL 数据库 4. Spring Boot 框架 5. Vue.js 前端框架			
三、实验方法（原理） 采用 SpringBoot 和 Vue 技术构建前后端分离的系统，题目可自行拟定，所实现的功能需涵盖但不限于以下方面： 1. 实现用户注册与登录功能 2. 完成对应信息的添加、删除、修改和查询操作 3. 实现文件上传功能 4. 做好安全管理工作 5. 确保界面具备友好性			

建议以 1 - 3 人组成一个小组，其中有 1 名或者 2 名前端开发人员以及 1 名或者 2 名后端开发人员。测试人员可由前端开发人员或者后端开发人员兼任，禁止只参与测试工作。

四、实验内容（步骤）

（一）项目概述（简述：项目背景、预期目标、主要功能、实现成果）

项目背景：随着企业规模扩大，传统纸质或 Excel 管理员工信息的方式效率低下、易出错、难以共享。为提升企业管理效率，亟需一套数字化员工管理系统，实现员工信息的集中化、规范化管理。

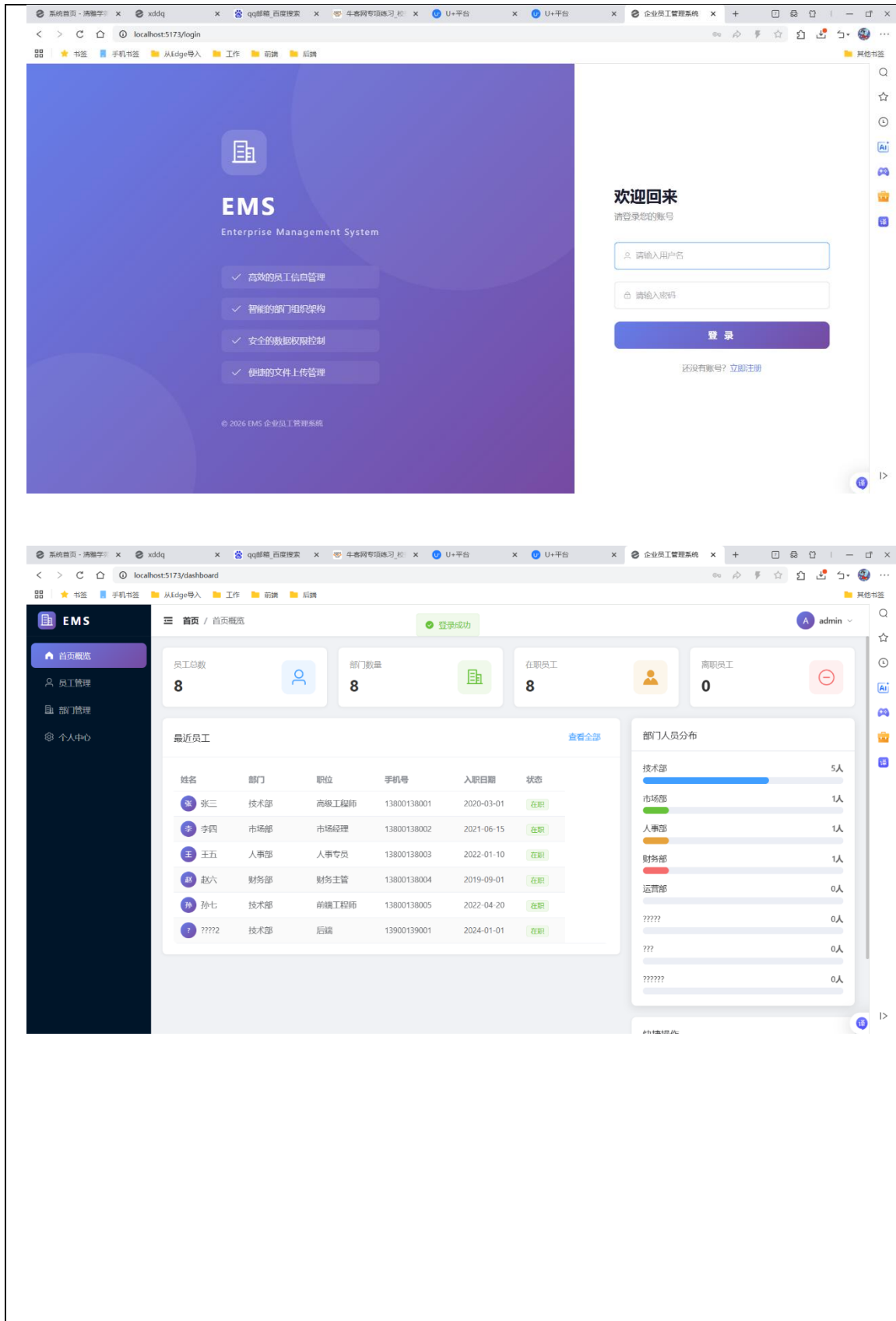
预期目标：构建一套基于 SpringBoot + Vue3 前后端分离架构的企业员工管理系统（EMS），实现用户认证、员工信息管理、部门管理、文件上传等核心功能，具备良好的用户体验和安全性。

主要功能：

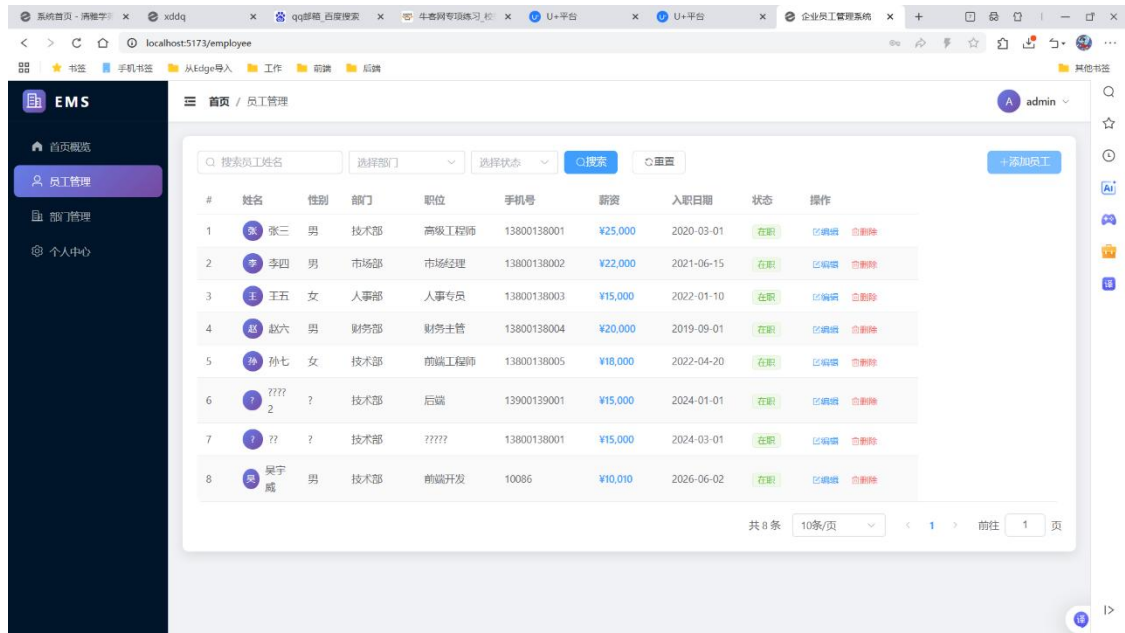
1. 用户注册与登录（JWT 令牌认证）
2. 员工信息 CRUD 及条件搜索
3. 部门信息 CRUD 及卡片式展示
4. 文件上传（头像管理）
5. 个人中心（资料修改、密码修改）
6. 首页仪表盘（数据统计可视化）

实现成果：成功开发并部署了功能完整的 EMS 系统，后端采用 Spring Boot 2.7.18 + MyBatis + Spring Security + JWT，前端采用 Vue 3 + Element Plus + Vite，数据库使用 MySQL 8.0，所有 22 个 API 接口测试通过。

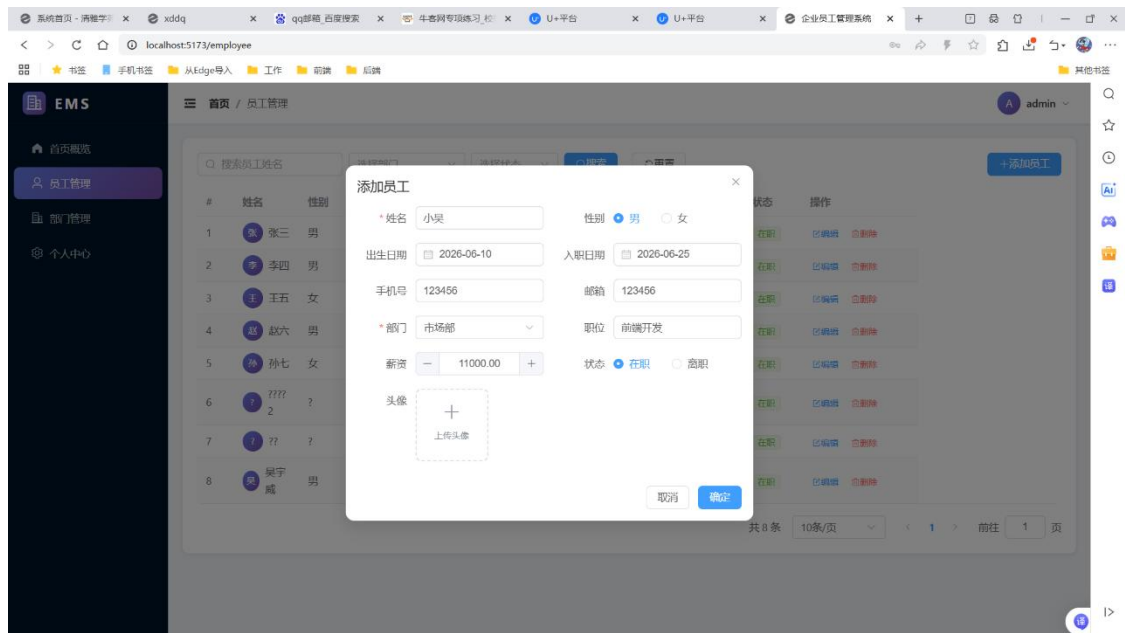
登录页：



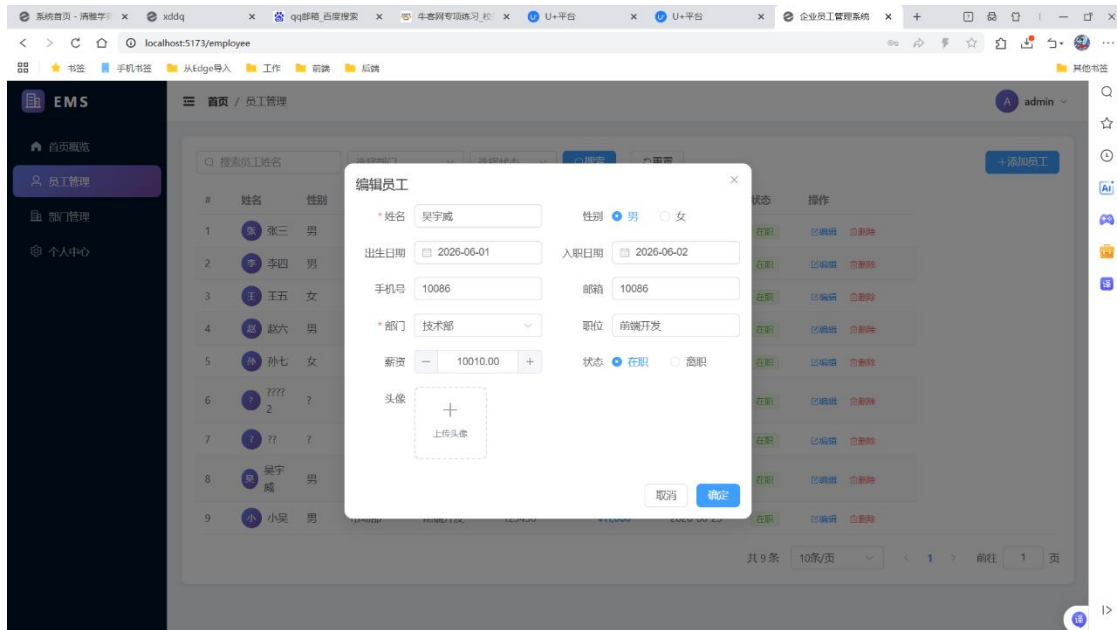
员工管理：



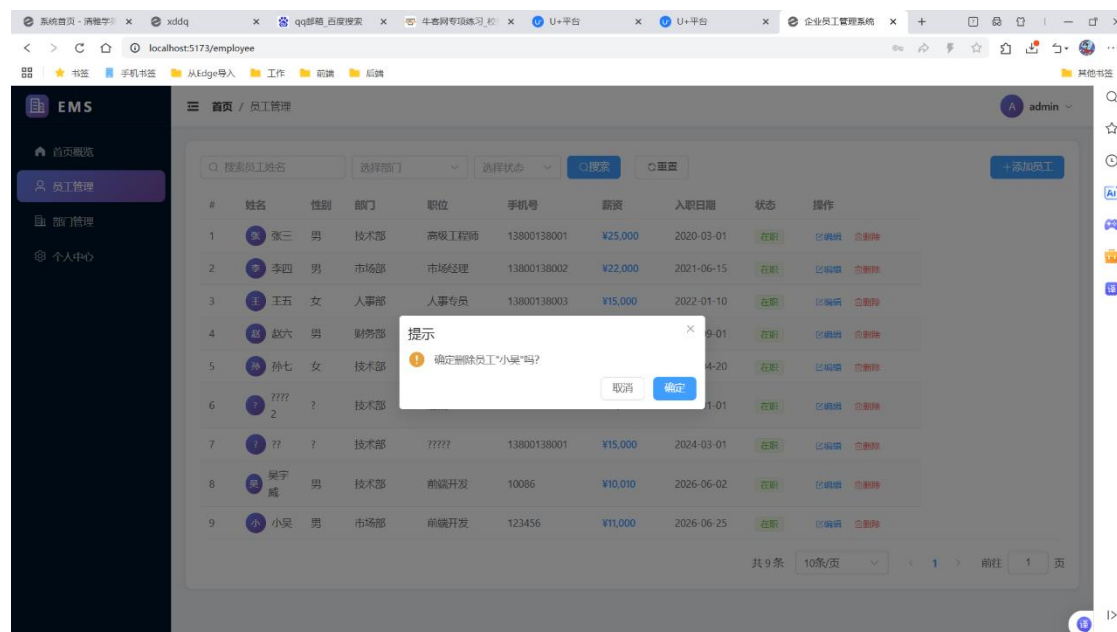
添加员工：



编辑员工：



删除员工：



员工搜索：

EMS 首页 / 员工管理 admin

搜索: 吴 选择部门 选择状态 搜索 重置 +添加员工

#	姓名	性别	部门	职位	手机号	薪资	入职日期	状态	操作
1	吴宁	男	技术部	前端开发	10086	¥10,010	2026-06-02	在职	详情 删除

共 1 条 10条/页 < 1 > 前往 1 页

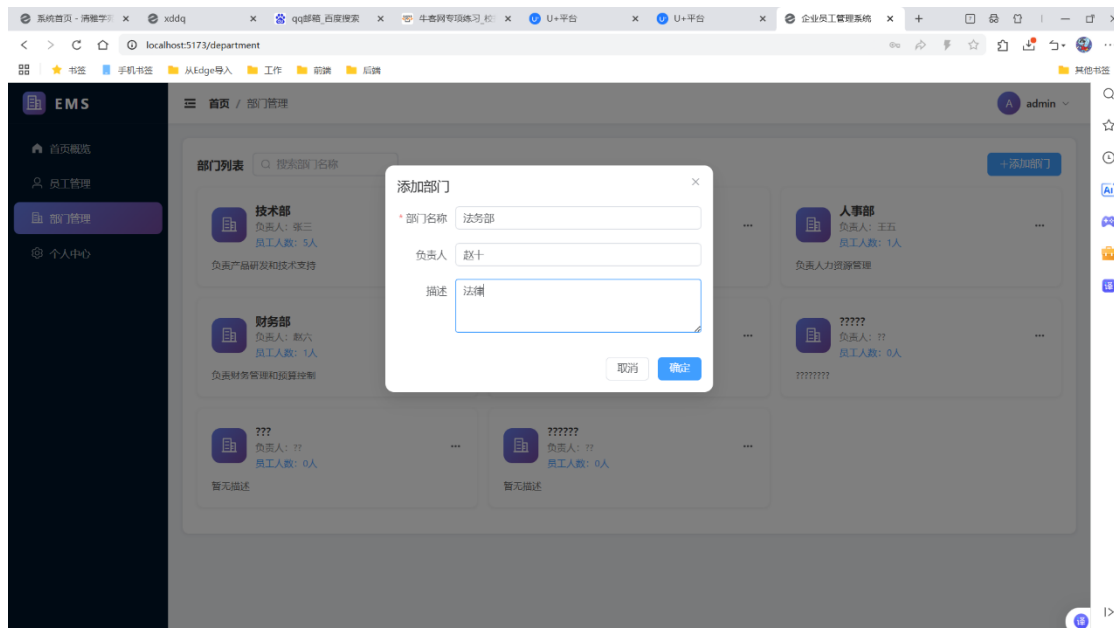
部门管理：

EMS 首页 / 部门管理 admin

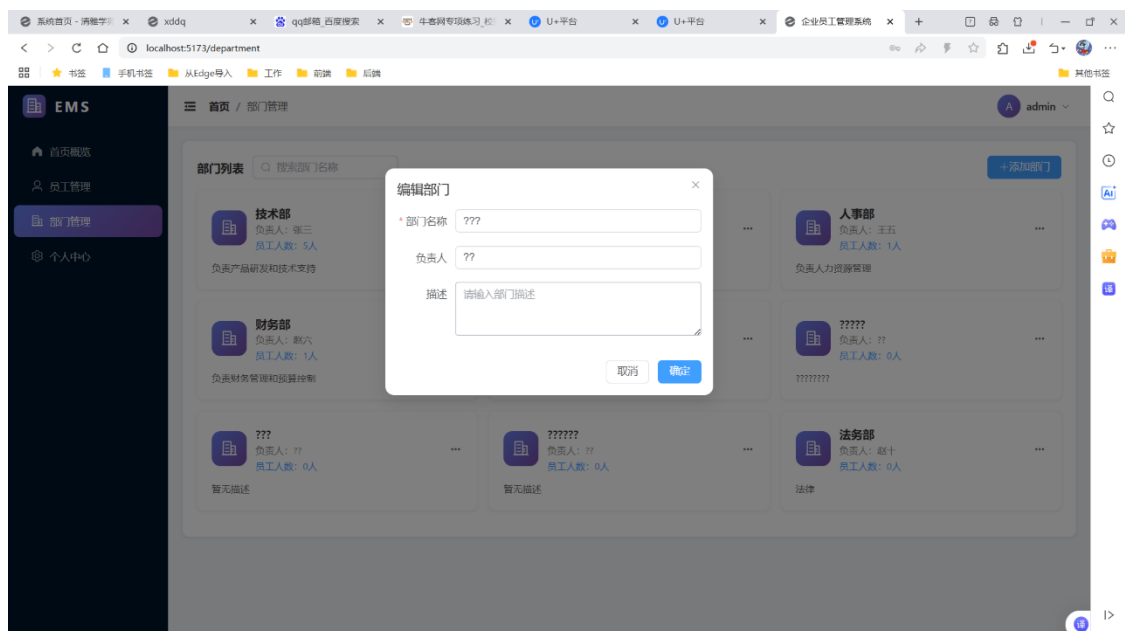
部门列表 搜索部门名称 +添加部门

- 技术部**
负责人: 张三
员工人数: 5人
负责产品研发和技术支持
- 市场部**
负责人: 李四
员工人数: 1人
负责市场推广和品牌建设
- 人事部**
负责人: 王五
员工人数: 1人
负责人力资源管理
- 财务部**
负责人: 赵六
员工人数: 1人
负责财务管理和预算控制
- 运营部**
负责人: 小七
员工人数: 0人
负责项目的运营
- ?????**
负责人: ??
员工人数: 0人
暂无描述
- ?????**
负责人: ??
员工人数: 0人
暂无描述

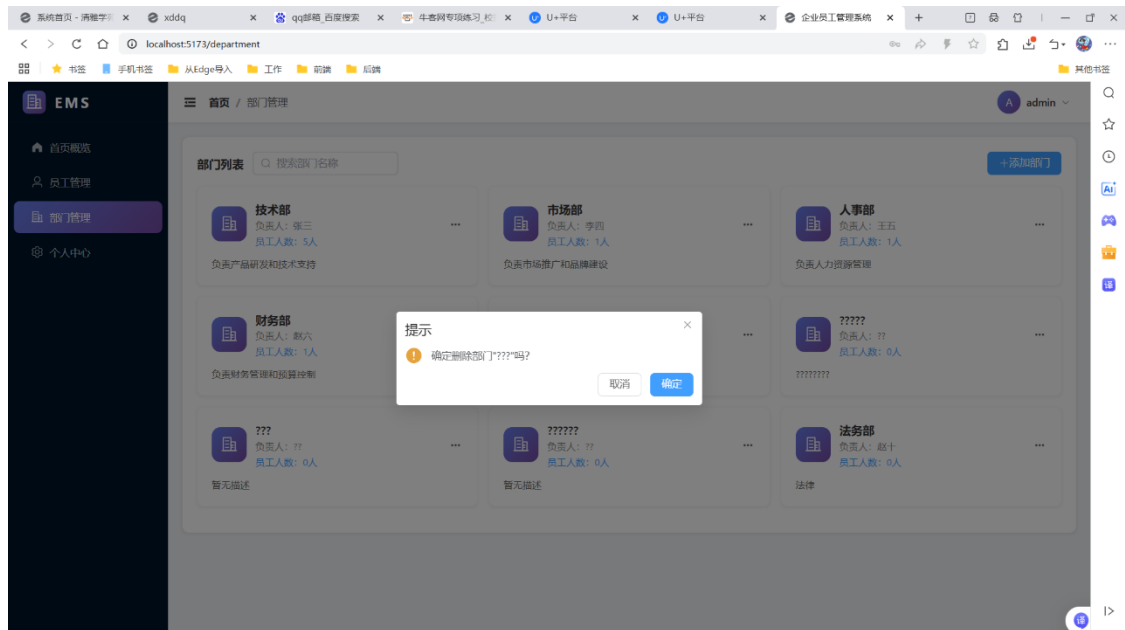
添加部门:



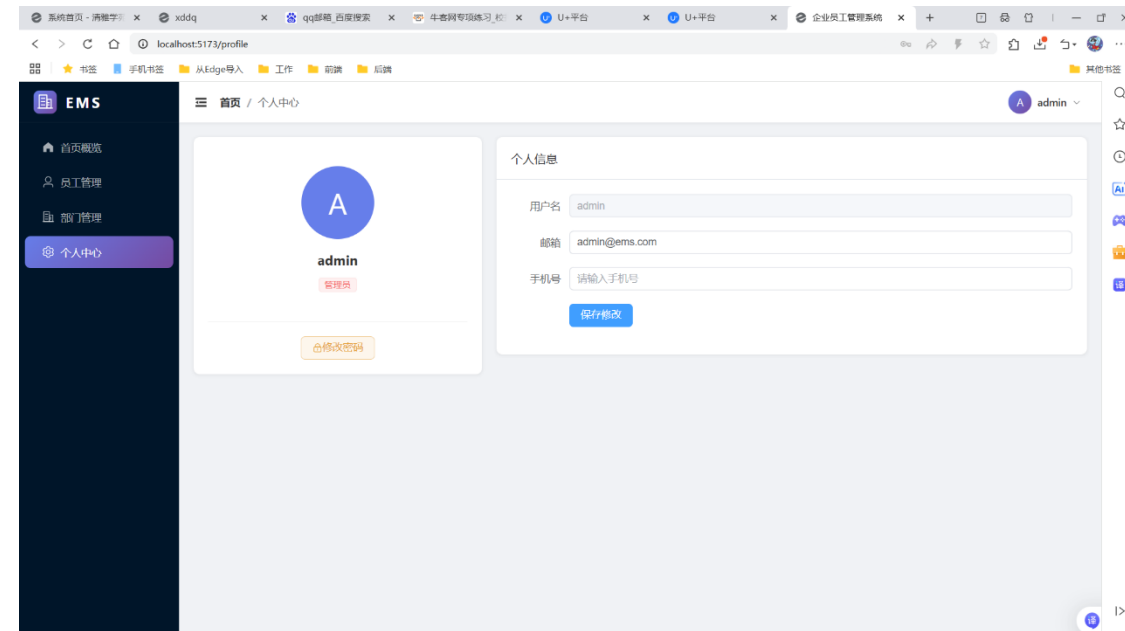
编辑部门:



删除部门：



个人中心：



(二) 项目需求分析

1. 功能需求（项目的主要功能模块）

模块	功能点	说明
用户认证	用户注册	新用户注册账号，密码 BCrypt 加密存储
	用户登录	JWT 令牌认证，返回 Token 和用户信息
	修改密码	验证旧密码后修改新密码
员工管理	员工列表	展示所有员工信息
	条件搜索	按姓名、部门、状态组合搜索
	添加员工	录入员工完整信息（含头像上传）
	编辑员工	修改员工信息
	删除员工	删除员工记录（含存在性校验）
部门管理	部门列表	卡片式展示所有部门
	添加部门	创建新部门
	编辑部门	修改部门信息
	删除部门	删除部门（含关联员工检查）
个人中心	资料修改	修改邮箱、手机号、头像
	头像上传	上传图片作为头像（限图片类型，5MB 以内）
仪表盘	数据统计	员工总数、部门数量、在职/离职统计
	部门分布	各部门人员占比可视化

2. 非功能需求（性能、安全、兼容性、可用性等需求）

安全性：采用 Spring Security + JWT 实现无状态认证，密码 BCrypt 加密存储，接口权限隔离（登录/注册公开，其余需认证），文件上传限制仅图片类型且不超过 5MB，防止权限提升攻击。

性能：前后端分离架构，Vite 开发服务器热更新，MyBatis 动态 SQL 按需查询，数据库索引优化（employee 表 department_id 索引）。

兼容性：前端响应式布局适配不同屏幕，支持 Chrome/Firefox/Edge 等主流浏览器。

可用性：Element Plus 组件库提供统一交互体验，表单校验即时反馈，全局

异常处理友好提示。

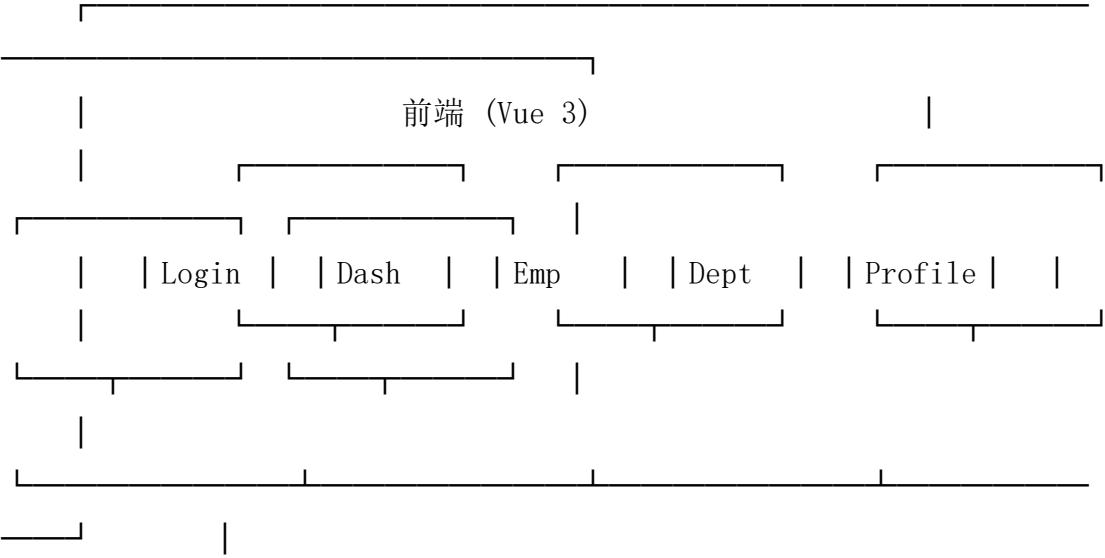
3. 技术需求（开发环境、数据库、第三方库、服务器等）

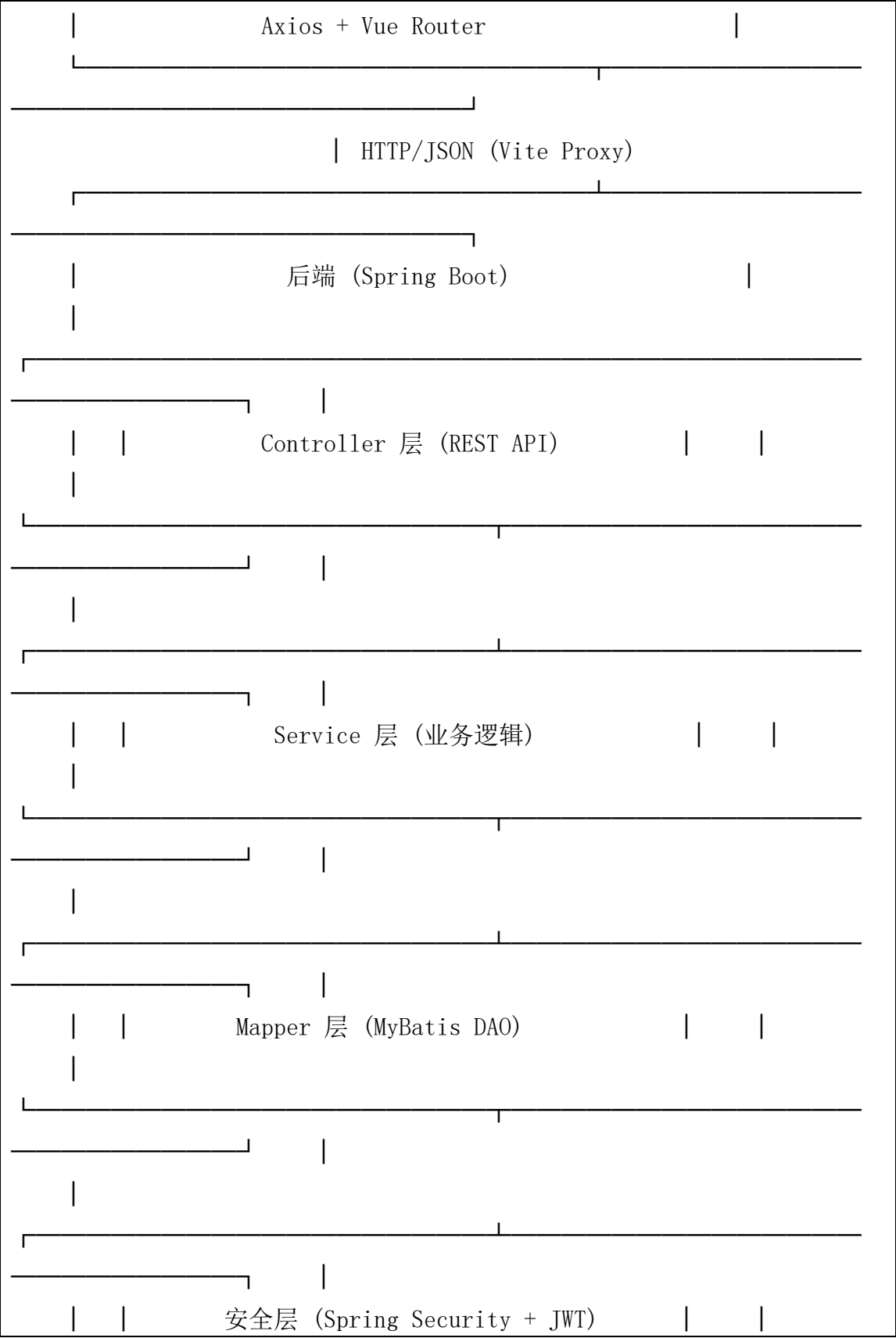
类别	技术	版本
后端框架	Spring Boot	2.7.18
ORM 框架	MyBatis	2.3.1
安全框架	Spring Security	2.7.x
认证方案	JWT (jjwt)	0.11.5
数据库	MySQL	8.0
前端框架	Vue 3	3.3.4
UI 组件库	Element Plus	2.3.14
构建工具	Vite	4.4.9
HTTP 客户端	Axios	1.5.0
路由	Vue Router	4.2.4
开发环境	JDK	1.8
构建工具	Maven	3.x

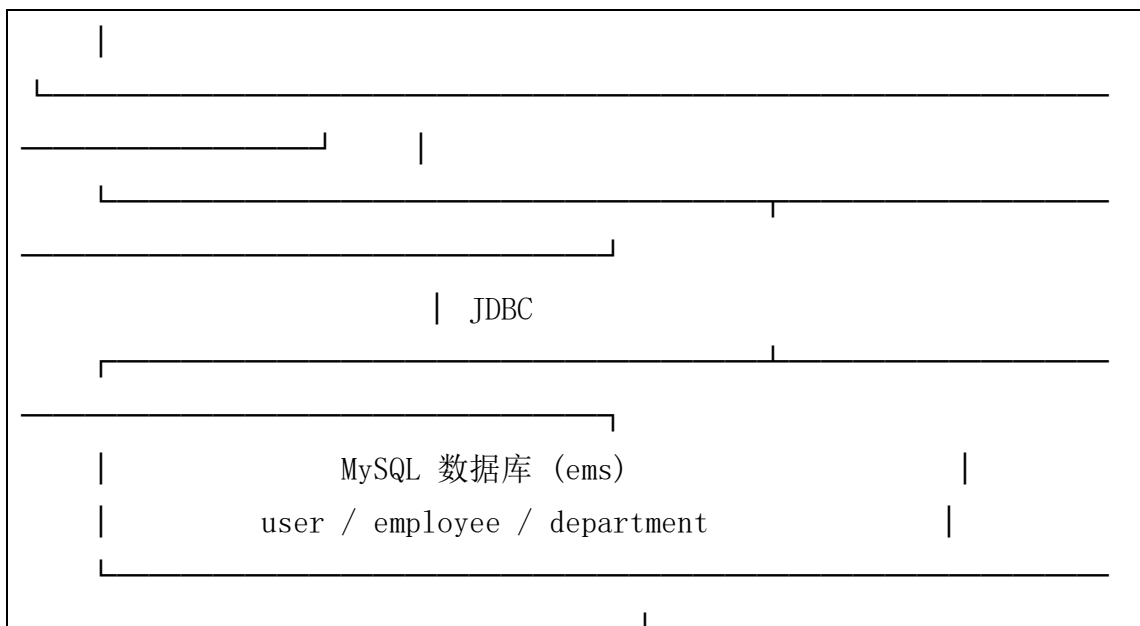
(三) 项目系统设计

1. 架构设计（系统整体架构设计、系统模块划分、模块间关系等）

系统采用 B/S 架构和前后端分离模式，整体架构如下：







模块间关系：Controllor 层接收 HTTP 请求，调用 Service 层处理业务；Service 层调用 Mapper 层操作数据库，返回 Result 统一响应；JwtAuthenticationFilter 拦截请求，验证 Token 后注入 Authentication；前端通过 Axios 发送请求，Vite 代理转发到后端 8088 端口。

2. 数据库设计（表结构设计、数据存储方案等）

user 表（用户表）：

字段	类型	说明
id	BIGINT	主键，自增
username	VARCHAR(50)	用户名，唯一索引
password	VARCHAR(100)	密码（BCrypt 加密）
email	VARCHAR(100)	邮箱
phone	VARCHAR(20)	手机号
avatar	VARCHAR(255)	头像路径
role	VARCHAR(20)	角色（user/admin）
status	TINYINT	状态（1 启用/0 禁用）
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

department 表（部门表）:

字段	类型	说明
id	BIGINT	主键，自增
name	VARCHAR(50)	部门名称
description	VARCHAR(200)	部门描述
manager	VARCHAR(50)	部门负责人
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

employee 表（员工表）:

字段	类型	说明
id	BIGINT	主键，自增
name	VARCHAR(50)	姓名
gender	VARCHAR(10)	性别
birth_date	DATE	出生日期
phone	VARCHAR(20)	手机号
email	VARCHAR(100)	邮箱
department_id	BIGINT	部门 ID（外键，索引）
position	VARCHAR(50)	职位
salary	DECIMAL(10, 2)	薪资
hire_date	DATE	入职日期
status	TINYINT	状态（1 在职/0 离职）
avatar	VARCHAR(255)	头像路径
create_time	DATETIME	创建时间
update_time	DATETIME	更新时间

（四）项目功能实现

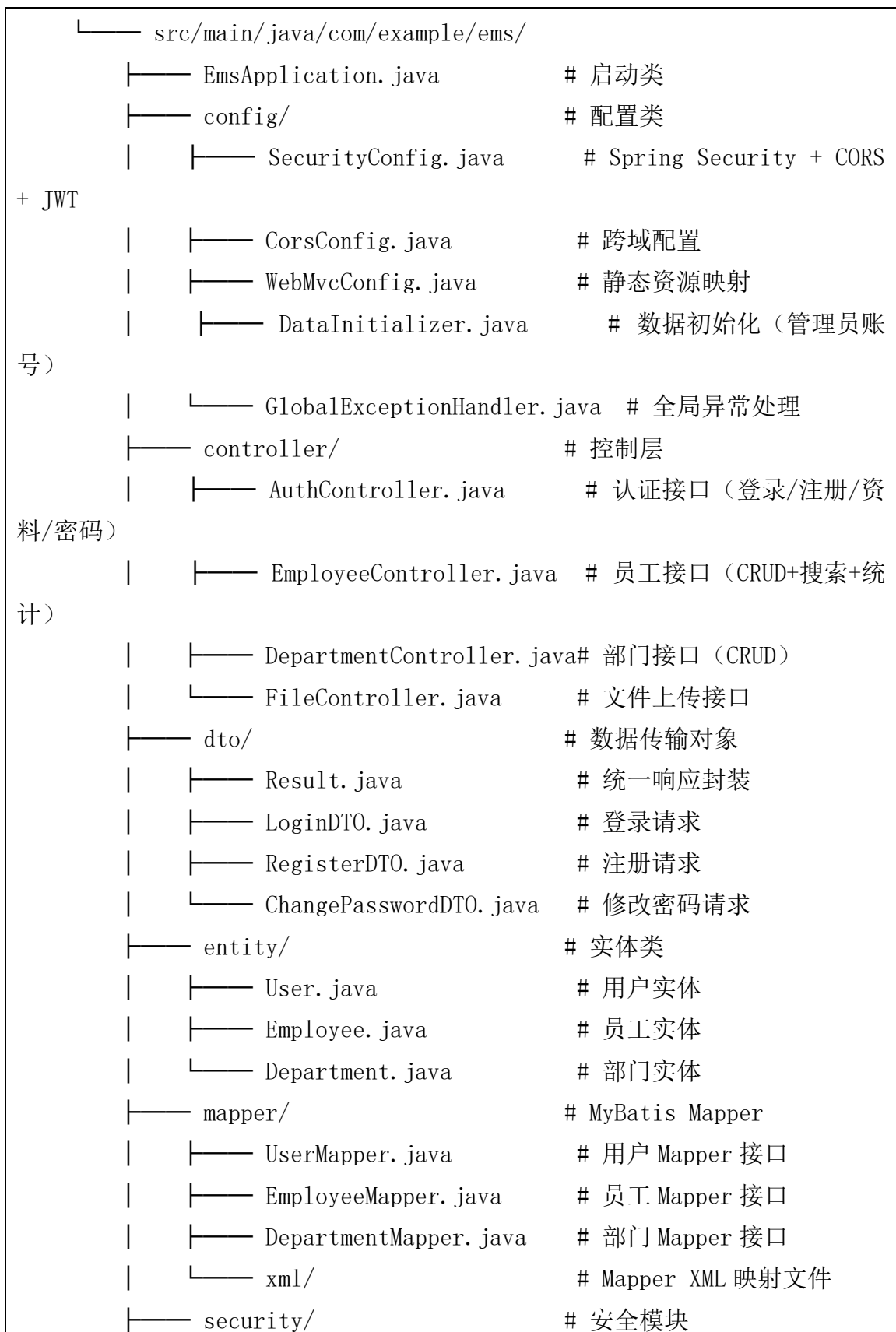
1. 项目结构说明(包含前端和后端)

后端结构:

backend/

└── pom.xml

Maven 依赖配置




```

|   |—— JwtTokenProvider.java    # JWT 令牌生成与解析
|   |—— JwtAuthenticationFilter.java # JWT 认证过滤器
|   |—— service/                  # 业务层
|       |—— UserService.java      # 用户业务
|       |—— EmployeeService.java  # 员工业务
|       |—— DepartmentService.java # 部门业务
|       |—— FileService.java      # 文件业务

```

前端结构:

```

frontend/
|—— package.json                # 依赖配置
|—— vite.config.js              # Vite 配置（代理）
|—— src/
    |—— main.js                 # 入口文件
    |—— App.vue                 # 根组件
    |—— styles/global.css       # 全局样式
    |—— router/index.js         # 路由配置
    |—— utils/request.js        # Axios 封装（拦截器）
    |—— api/                   # API 接口封装
        |—— auth.js             # 认证 API
        |—— employee.js         # 员工 API
        |—— department.js       # 部门 API
        |—— file.js             # 文件 API
    |—— views/                 # 页面组件
        |—— Login.vue           # 登录页
        |—— Register.vue        # 注册页
        |—— Layout.vue          # 布局框架（侧边栏+头部）
        |—— Dashboard.vue       # 首页仪表盘
        |—— Employee.vue        # 员工管理
        |—— Department.vue      # 部门管理
        |—— Profile.vue         # 个人中心

```

2. 主要功能实现和关键代码呈现

(1) JWT 认证机制

JWT 令牌生成 (JwtTokenProvider.java):

```
public String generateToken(Long userId, String username, String
role) {
    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + expiration);
    return Jwts.builder()
        .setSubject(String.valueOf(userId))
        .claim("username", username)
        .claim("role", role)
        .setIssuedAt(now)
        .setExpiration(expiryDate)
        .signWith(getSigningKey(), SignatureAlgorithm.HS256)
        .compact();
}
```

JWT 认证过滤器 (JwtAuthenticationFilter.java) 拦截每个请求, 从 Header 提取 Token 并验证:

```
@Override
protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain filterChain) {
    String token = resolveToken(request);
    if (token != null && jwtTokenProvider.validateToken(token)) {
        Long userId = jwtTokenProvider.getUserIdFromToken(token);
        String username =
            jwtTokenProvider.getUsernameFromToken(token);
        String role =
            jwtTokenProvider.parseToken(token).get("role", String.class);
        // 构建 Authentication 对象并设置到 SecurityContext
        UsernamePasswordAuthenticationToken auth =
```

```

        new UsernamePasswordAuthenticationToken(userId, null,
authorities);

SecurityContextHolder.getContext().setAuthentication(auth);
    }
    filterChain.doFilter(request, response);
}

```

(2) Spring Security 安全配置

SecurityConfig.java 配置了 CORS、CSRF 禁用、无状态会话、路径权限：

```

http.cors().and()
    .csrf().disable()
    .sessionManagement().sessionCreationPolicy(SessionCreationPol
icity.STATELESS)
    .and()
    .authorizeRequests()
        .antMatchers("/api/auth/login",
"/api/auth/register").permitAll()
        .antMatchers(HttpMethod.GET, "/uploads/**").permitAll()
        .anyRequest().authenticated()
    .and()
    .addFilterBefore(jwtAuthenticationFilter,
        UsernamePasswordAuthenticationFilter.class);

```

(3) 统一响应封装

Result.java 统一所有 API 的返回格式：

```

@Data
public class Result<T> {
    private int code;
    private String message;
    private T data;

    public static <T> Result<T> success(T data) {
        Result<T> r = new Result<>();

```

```

        r.code = 200; r.message = "操作成功"; r.data = data;
        return r;
    }

    public static <T> Result<T> error(String message) {
        Result<T> r = new Result<>();
        r.code = 500; r.message = message; r.data = null;
        return r;
    }
}

```

(4) MyBatis 动态 SQL 条件查询

EmployeeMapper.xml 实现灵活的条件搜索:

```

<select id="findByCondition" resultMap="employeeResultMap">
    SELECT e.*, d.name as department_name
    FROM employee e LEFT JOIN department d ON e.department_id =
d.id
    <where>
        <if test="name != null">
            AND e.name LIKE CONCAT('%', #{name}, '%')
        </if>
        <if test="departmentId != null">
            AND e.department_id = #{departmentId}
        </if>
        <if test="status != null">
            AND e.status = #{status}
        </if>
    </where>
    ORDER BY e.create_time DESC
</select>

```

(5) 前端 Axios 拦截器

request.js 统一处理 Token 注入和错误响应:

```

request.interceptors.request.use(config => {

```

```

    const token = localStorage.getItem('token')
    if (token) config.headers.Authorization = `Bearer ${token}`
    return config
  })

request.interceptors.response.use(response => {
  const res = response.data
  if (res.code !== 200) {
    ElMessage.error(res.message || '请求失败')
    if (res.code === 401) {
      localStorage.removeItem('token')
      router.push('/login')
    }
    return Promise.reject(new Error(res.message))
  }
  return res
})

```

(6) 文件上传安全校验

FileService.java 限制文件类型和大小:

```

public Result<?> upload(MultipartFile file) {
  String contentType = file.getContentType();
  if (contentType == null || !contentType.startsWith("image/"))
  {
    throw new RuntimeException("仅支持上传图片文件");
  }
  if (file.getSize() > 5 * 1024 * 1024) {
    throw new RuntimeException("文件大小不能超过 5MB");
  }
  // 生成唯一文件名并保存
}

```

3. 重难点功能实现的解决方案(若有则写, 若无可删除)

(1) Spring Security + JWT 无状态认证

难点: Spring Security 默认基于 Session, 需改为 JWT 无状态认证。解决方案: 禁用 Session (SessionCreationPolicy.STATELESS), 自定义 JwtAuthenticationFilter 在 UsernamePasswordAuthenticationFilter 之前执行, 从请求头提取 Token 验证后注入 Authentication 对象。

(2) CORS 跨域与 Spring Security 集成

难点: 前端 5173 端口访问后端 8088 端口存在跨域问题, 且 Spring Security 会在 CORS Filter 之前拦截 OPTIONS 预检请求。解决方案: 在 SecurityConfig 中添加.cors().and() 让 Spring Security 先处理 CORS, 同时配置 CorsFilter 允许前端域名访问。

(3) 权限提升漏洞防护

难点: updateProfile 接口如果直接接收完整 User 对象, 攻击者可通过修改 role 字段提权为 admin。解决方案: updateProfile 方法只允许更新 email、phone、avatar 三个安全字段, 忽略 role、status、password 等敏感字段。

(4) 密码修改持久化

难点: MyBatis 动态 SQL 的<update>语句中 password 字段缺失, 导致修改密码后数据库未更新。解决方案: 在 UserMapper.xml 的<set>块中添加<if test="password != null">password = #{password},</if>。

(五) 项目功能测试与调试 (包含所有的功能演示)

序号	测试功能	测试方法	预期结果	实
1	管理员登录	POST /api/auth/login, 账号 admin/admin123	返回 code=200, 含 token 和用户信息	通过
2	用户注册	POST /api/auth/register, 新用户 testuser2	返回 code=200, 注册成功	通过
3	新用户登录	POST /api/auth/login, 账号 testuser2/test123456	返回 code=200, 含 token	通过

4	修改密码	PUT /api/auth/password, 旧密码+新密码	返回 code=200, 密码修改成功	通过	
5	新密码登录	POST /api/auth/login, 使用新密码	返回 code=200, 登录成功	通过	
6	获取用户信息	GET /api/auth/info, 带 Token	返回用户信息	通过	
7	修改个人资料	PUT /api/auth/profile, 修改邮箱和手机号	返回 code=200, 资料更新	通过	
8	员工列表查询	GET /api/employee/list	返回所有员工列表	通过	
9	条件搜索员工	GET /api/employee/list?name=张三	返回匹配的员工	通过	
10	添加员工	POST /api/employee, 完整员工信息	返回 code=200	通过	
11	编辑员工	PUT /api/employee, 修改薪资	返回 code=200	通过	
12	删除员工	DELETE /api/employee/{id}	返回 code=200	通过	
13	员工统计	GET /api/employee/stats	返回 totalEmployees 和 activeEmployees	通过	
14	部门列表查询	GET /api/department/list	返回所有部门	通过	
15	添加部门	POST /api/department	返回 code=200	通过	
16	编辑部门	PUT /api/department	返回 code=200	通过	
17	删除部门	DELETE /api/department/{id}	返回 code=200	通过	
18	未认证访问拦截	GET /api/auth/info, 不带 Token	返回 401	通过	
19	密码修改接口拦截	PUT /api/auth/password, 不带 Token	返回 401	通过	
20	删除不存在员工	DELETE /api/employee/99999	返回错误“员工不存在”	通过	
21	文件上传	POST /api/file/upload, 图片文件	返回文件 URL	通过	
22	非图片上传拦截	POST /api/file/upload, 非图片文件	返回错误提示	通过	

五、结果分析

通过本次 SpringBoot 企业级开发实验，我系统掌握了前后端分离项目的完整开发流程，从需求分析、系统设计到编码实现和测试部署，每个环节都有了深

入的理解和实践。

在技术层面，我深入学习了 Spring Boot 框架的核心特性，包括自动配置、依赖注入和 Starter 机制。掌握了 Spring Security 安全框架的配置方法，实现了基于 JWT 的无状态认证方案，理解了 SecurityFilterChain 的工作原理和过滤器的执行顺序。MyBatis 的动态 SQL 让我体会到了灵活查询的便利性，`<if>`、`<where>`等标签可以根据参数动态拼接 SQL，避免了硬编码。

在前后端分离架构方面，我理解了 Vue 3 组合式 API 的优势，Element Plus 组件库大大提升了开发效率。Axios 拦截器统一处理 Token 注入和错误响应，Vite 代理解决了开发环境的跨域问题。前后端通过 JSON 格式交互，接口文档的规范性直接影响联调效率。

开发过程中遇到了多个关键问题并逐一解决：一是 Spring Security 的 CORS 配置必须使用 `.cors().and()` 而非仅靠 `CorsFilter`，否则预检请求会被拦截；二是 MyBatis 的 `<update>` 语句必须包含 `password` 字段，否则密码修改无法持久化；三是 `updateProfile` 接口必须过滤敏感字段，防止权限提升攻击。这些问题让我认识到安全开发的细节重要性。

今后我需要在以下方面加强学习：一是微服务架构，将单体应用拆分为独立服务；二是 Redis 缓存提升查询性能；三是 Docker 容器化部署和 CI/CD 自动化；四是更深入的安全防护（SQL 注入防御、XSS 防护、接口限流等）。本次实验为我打下了扎实的基础，后续将持续提升企业级开发能力。

项目开发团队成员及分工表

序号	学号	姓名	任务分工
01	202516060116	黄显杰	后端开发 (Spring Boot + MyBatis + Security + JWT)、数据库设计
02	202516060143	吴宇威	前端开发 (Vue 3 + Element Plus + Vite)、UI 设计与页面交互
03			

六、成绩评定

评 语：

指导教师签字：